UN Voting

```
#| echo: true
#| warning: false
#| message: false

library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```
Warning: package 'purrr' was built under R version 4.5.2
```

```
Warning: package 'dplyr' was built under R version 4.5.2
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   4.0.3     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.2.2
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```
library(data.table)
```

```
Attaching package: 'data.table'
```

```
The following objects are masked from 'package:lubridate':
```

```
    hour, isoweek, mday, minute, month, quarter, second, wday, week,
    yday, year

The following objects are masked from 'package:dplyr':

    between, first, last

The following object is masked from 'package:purrr':

    transpose
```

```
library(igraph)
```

```
Warning: package 'igraph' was built under R version 4.5.2


Attaching package: 'igraph'

The following objects are masked from 'package:lubridate':

    %--%, union

The following objects are masked from 'package:dplyr':

    as_data_frame, groups, union

The following objects are masked from 'package:purrr':

    compose, simplify

The following object is masked from 'package:tidyr':

    crossing

The following object is masked from 'package:tibble':

    as_data_frame

The following objects are masked from 'package:stats':

    decompose, spectrum
```

```
The following object is masked from 'package:base':

    union
```

```r
# 1. Load the data using your original method
votes_data <- read_csv(file.path("~/Desktop", "Voting by UN.csv"))
```

```
Rows: 916554 Columns: 20
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (14): ms_code, ms_name, ms_vote, date, session, resolution, draft, commi...
dbl  (6): undl_id, total_yes, total_no, total_abstentions, total_non_voting,...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# 2. Extract the year safely
votes_data <- votes_data %>%
  mutate(
    year_text = sub(".*/", "", date),
    year_num = as.numeric(year_text),
    year = ifelse(year_num < 100, year_num + 2000, year_num)
  )

# Define the exact list of countries you want to analyze
target_countries <- c(
  "UNITED STATES",
  "GERMANY",
  "CHINA",
  "RUSSIA",
  "JAPAN",
  "UNITED KINGDOM",
  "CANADA",
  "SPAIN",
  "MEXICO",
  "INDIA",
  "AUSTRALIA")

# 3. Filter for Y votes, years 2000-2026, AND your target countries
y_votes <- votes_data %>%
  dplyr::filter(ms_vote == "Y") %>%
```

```
  dplyr::filter(year >= 2000 & year <= 2026) %>%
  dplyr::filter(tolower(ms_name) %in% tolower(target_countries) |
                grepl("United States|United Kingdom|Russian|Korea", ms_name, ignore.case = TR
  select(ms_name, resolution, date) %>%
  # === ADDED CLEANING STEP: Standardize the long names to your exact targets ===
  mutate(ms_name = case_when(
    grepl("Russia", ms_name, ignore.case = TRUE) ~ "RUSSIA",
    grepl("Korea", ms_name, ignore.case = TRUE)  ~ "SOUTH KOREA",
    grepl("United States|USA", ms_name, ignore.case = TRUE) ~ "UNITED STATES",
    grepl("United Kingdom|UK", ms_name, ignore.case = TRUE) ~ "UNITED KINGDOM",
    TRUE ~ toupper(ms_name) # Makes all other countries uppercase to match your style
  ))

# ======================================================================
# HIGH-PERFORMANCE STEP: data.table engine
# ======================================================================
setDT(y_votes)

counted_pairs <- y_votes[y_votes,
                         on = .(resolution, date),
                         allow.cartesian = TRUE][
                           ms_name != i.ms_name,
                           .N,
                           by = .(ms_name, i.ms_name)]

setnames(counted_pairs, c("ms_name", "i.ms_name", "N"), c("ms_name_A", "ms_name_B", "n"))

rm(y_votes, votes_data)
gc()
```

```
         used (Mb) gc trigger  (Mb) limit (Mb) max used  (Mb)
Ncells 1428802 76.4    2497975 133.5         NA  2497975 133.5
Vcells 2516506 19.2   40036568 305.5      16384 50045702 381.9
```

```
# ======================================================================
# 5. Build your adjacency matrix from the memory-safe counted pairs
# ======================================================================
adj_matrix <- counted_pairs %>%
  pivot_wider(names_from = ms_name_B, values_from = n, values_fill = 0) %>%
  column_to_rownames(var = "ms_name_A") %>%
  as.matrix()
```

```r
# 6. Turn the matrix into an igraph object
n <- graph_from_adjacency_matrix(
  adj_matrix,
  mode = "undirected",
  weighted = TRUE,
  diag = FALSE
)
```

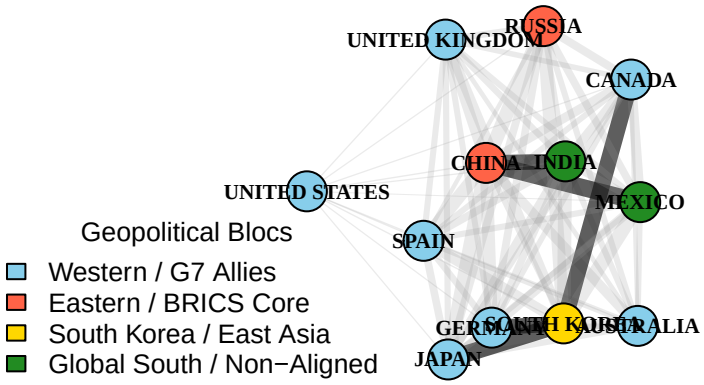
```
Warning: The `adjmatrix` argument of `graph_from_adjacency_matrix()` must be symmetric
with mode = "undirected" as of igraph 1.6.0.
i Use mode = "max" to achieve the original behavior.
```

```r
summary(n)
```

```
IGRAPH 256f6ab UNW- 12 66 --
+ attr: name (v/c), weight (e/n)
```

```r
# ======================================================================
# 7. HIGH-IMPACT GEOPOLITICAL BLOC COLORING & CUSTOM KEY
# ======================================================================
# Instead of random colors, let's manually group them logically:
# Group 1 (Blue): Western/G7 Allies (US, UK, Canada, Germany, Spain, Japan, Australia)
# Group 2 (Red): BRICS Core Counterweights (China, Russia)
# Group 3 (Green): Non-Aligned / Rising Regional Powers (India, Mexico)

# Create a clean data frame to map the countries to colors
bloc_mapping <- data.frame(
  name = c("UNITED STATES", "UNITED KINGDOM", "CANADA", "GERMANY", "SPAIN", "JAPAN", "AUSTRA
           "CHINA", "RUSSIA",
           "SOUTH KOREA", "INDIA", "MEXICO"),
  bloc_color = c("skyblue", "skyblue", "skyblue", "skyblue", "skyblue", "skyblue", "skyblue"
                 "tomato", "tomato",
                 "gold", "forestgreen", "forestgreen")
)

# Match the colors onto the network nodes
v_names <- vertex_attr(n, "name")
node_colors <- bloc_mapping$bloc_color[match(v_names, bloc_mapping$name)]
n <- set_vertex_attr(n, "color", value = node_colors)
```

```r
# --- EXTRACT AND HYPER-SCALE EDGE STRENGTHS ---
country_labels <- vertex_attr(n, "name")
raw_weights <- edge_attr(n, "weight")

# Mathematical normalization: Subtract the minimum weight and scale up
# This makes weak ties paper-thin and strong alliances thick and bold
min_w <- min(raw_weights)
max_w <- max(raw_weights)
scaled_edge_widths <- ((raw_weights - min_w) / (max_w - min_w)) * 8 + 0.5

# Highlight top-tier relationships with a darker color, make others translucent
edge_colors <- ifelse(raw_weights > (max_w - (max_w - min_w)*0.15),
                      rgb(0.1, 0.1, 0.1, 0.7),   # Dark gray for strongest ties
                      rgb(0.6, 0.6, 0.6, 0.2))  # Faint gray for standard ties

# ======================================================================
# 8. Plot the Analytical Graph with a Legend (Key)
# ======================================================================
# Increase plot margins slightly to fit the legend cleanly
par(mar = c(4, 1, 3, 1))

plot(n,
  layout = layout_with_fr,                # Keeps factions clearly separated
  vertex.size = 24,                       # Easy to read node sizes
  vertex.label = country_labels,
  vertex.label.cex = 0.7,                # Bold, sharp text labels
  vertex.label.font = 2,
  vertex.label.color = "black",
  edge.width = scaled_edge_widths,         # Maps true alignment to line thickness
  edge.color = edge_colors,                # Highlights the absolute strongest alliances
  main = "UN Voting Alignment Factions (2000-2026)\nRelative Relationship Strength"
)

# Add the Key (Legend) to the bottom left corner
legend("bottomleft",
       legend = c("Western / G7 Allies", "Eastern / BRICS Core", "South Korea / East Asia", "
       fill = c("skyblue", "tomato", "gold", "forestgreen"),
       bty = "n",
       cex = 0.8,
       title = "Geopolitical Blocs")
```




```
# ======================================================================
# 9. PRINT TEXT CALLOUTS FOR UNDERSTANDING RELATIONSHIP AMOUNTS
# ======================================================================
cat("\n=========================================================\n")
```

```
=========================================================
```

```
cat("   TOP 5 STRONGEST VOTING ALLIANCES (HIGHEST OVERLAP)   \n")
```

```
   TOP 5 STRONGEST VOTING ALLIANCES (HIGHEST OVERLAP)
```

```
cat("=========================================================\n")
```

```
=========================================================
```

```
top_allies <- counted_pairs %>%
  dplyr::filter(ms_name_A < ms_name_B) %>% # Removes duplicate combinations (A-B vs B-A)
  arrange(desc(n)) %>%
  head(5)

print(top_allies)
```

```
   ms_name_A   ms_name_B     n
      <char>      <char> <int>
1:    MEXICO SOUTH KOREA  2476
2:     CHINA SOUTH KOREA  2385
3:     INDIA SOUTH KOREA  2271
4:    RUSSIA SOUTH KOREA  2113
5:     JAPAN SOUTH KOREA  2043
```

“‘

The `echo: false` option disables the printing of code (only output is displayed).